

GUI Design

Overview

I was able to recycle much of the code from Assignment 1. Because I implemented the UI separately by following the MVC pattern, this meant that I just needed to re-implement the UI interface to use a GUI instead of the console, which meant several methods needed minimal changes. One issue I faced was that the old interface did not allow for event driven interaction, so I had to change some of the core Cluedo code to make the GUI work. This meant that the TextUI interface no longer works, which was disappointing but is necessary.

Main GUI

The Main GUI has two parts to it, the display of the board and everything on it, and the control panel. This was achieved by having two objects added to the main JFrame, a GUICanvas which extends Canvas for the board and a JPanel for the control panel at the bottom section of the GUI. It also has a JMenu above the



board, from which the detective notes and cards of the player can be viewed, and the game can also be exited from this menu.

The JFrame and the control panel does not use a LayoutManager i.e. it uses an Absolute Layout, which allowed for more flexibility in positioning the components than a BorderLayout or a FlowLayout.

The panel has four main sections: the display of the dice roll value and the remaining move steps on the left, the display of the information for players (in particular, the player who's turn it is currently) in the centre, the buttons to open the information windows on the right and bottom centre, and the status bar for mouse hover and invalid moves info at the bottommost section.

When the frame is resized, the Square's height and width needs to be recomputed and redrawn. In this case, the Square's sizes/dimensions are uniformed which meant that I only have to re-compute once.

The GUI also acts as a KeyListener to act to shortcut keys implemented to enable experienced users to operate the program more efficiently.

InputOutput (IO)

The IO class handles the input and output of various components that are not directly associated with the main GUI.

An example of this would be the YourCards and CardsSeen buttons which brings up a dialog that displays a list of Cards when clicked. For the YourCards dialog, these each 'bullet' represents one of the cards the player was dealt at the start of the game. For the CardsSeen dialog each 'bullet' corresponds to one of the cards the player has seen during the course of the game. Initially these are all of the cards the player was dealt, but as the game progresses and the player has some suggestions proven false; various cards are added to this set.

When the game is started, several dialog boxes are shown. The first is a welcome message, the second asks the user how many human players the game should have, and the rest ask each player which character they would like to control for the game. Players are not able to select characters that have been chosen by other players already.

At several other points during the game, dialog boxes are shown to gather information from the player. The most notable of these is when a player is making a suggestion. Two dialog boxes are shown, the first has a list of all the characters in the game and asks the user to choose which one they think was involved in the murder, and the second has a list of weapons for the user to choose the murder weapon from. For an accusation, there are three dialog boxes shown to account for the room

Dialog message boxes were also used to display messages to the user as they played the game, such as which card they have been shown after their suggestion (if any), and to display if an accusation made was correct or not. These did not take any input from the user, just simply displayed a message.

The Board and GUICanvas

To draw the board, I used a GUICanvas object. Firstly, the board image was drawn at position (0,0), and then all of the token on the board were drawn on top of the board image. The positioning of these tokens was determined by a grid overlay implemented, which lined up with the positioning of the tiles on the board image. Additionally, the GUICanvas can also control the output of the status bar when the mouse hovers on the board.

When a user clicks on the board, the first thing that occurs is that the grid position that they click is determined, by using the grid overlay made. Secondly, I need to work out if the player was able to move their piece on that row,col position (which has to be a neighbor of their current location). The 'movable' Squares were shown by adding an Oval with 'thickened' border to the Square (including Room Squares).

If the player is allowed to move to the position they selected, their token is animated from their current location to their new location, then they are moved to that new location. If they are in a room, they can make a suggestion. Additionally, the game also needs to remember if a Player has been to a particular Square or set of Squares in one given turn such that a Player cannot go back to the original location they are in at the start of their turn

On the other hand, if the player cannot move to the Square they selected, a status bar message appears indicating that it is an invalid move, and the user has to select a Square that it can legally move to.